

Interview Feedback Platform

Companion diagrams and interface design for `ia-documentation-v1.md` — built with the reference document's format as a benchmark: labelled user interfaces, a UI flow overview, an ER diagram, data flow & flowchart notation, a UML class diagram, and validation tests.

● Admin

● Interviewer

01 · CRITERION A

Rationale for Proposed Solution

A client–server web application is the right architectural pattern for this problem because the system naturally splits along role boundaries — an admin configuring positions and questions, an interviewer consuming and scoring them — that map cleanly onto separate, reusable interface and route modules. Keeping business rules (session checks, score validation, atomic submission) on the server means every client sees the same enforced behaviour, rather than duplicating validation logic across screens.

My choice of language for the backend is **Python**, using the **FastAPI** framework, because of the strength of its ecosystem for exactly this kind of data-driven service: SQLAlchemy for structured, typed database access, Pydantic for request/response validation that rejects malformed input before it reaches business logic, and passlib for password hashing rather than rolling my own. I chose **Visual Studio Code** as my IDE for this project, as its integrated debugger, inline type-checking, and REST client extensions let me step through request handling and inspect state directly, rather than debugging through print statements.

My choice of database management system is **PostgreSQL**, as it is a mature, open-source relational database that enforces the data integrity this system depends on directly at the schema level — a CHECK constraint makes an out-of-range score physically impossible to store, and foreign keys guarantee a score can never reference a question that doesn't belong to the candidate's position. Through JOIN queries across candidates, questions, and

interview_scores, the admin can retrieve a candidate's full scoring breakdown in a single query, and its transaction support is what makes the "submit all scores atomically" requirement actually guaranteed rather than merely likely.

Success Criteria

- 1 The admin should be able to log in with a username and password.
- 2 The interviewer should be able to log in with a username and password and see only the candidates assigned to them.
- 3 The admin should be able to create a Position and add an ordered list of questions to it.
- 4 The admin should be able to create a Candidate record and assign it to exactly one Position and one Interviewer.
- 5 The interviewer should be able to view an assigned candidate's questions in the exact order set by the admin.
- 6 The interviewer should be able to enter a score from 1–5 and an optional comment for every question.
- 7 The interviewer should be able to submit all scores for a candidate in a single action, after which the candidate is marked completed.
- 8 The system should reject any score entered outside the 1–5 range.
- 9 A completed candidate should no longer appear in the interviewer's active queue.
- 10 All data entered should remain stored correctly after the server restarts.

Every screen a user can reach from login, grouped by the module and role that owns it.

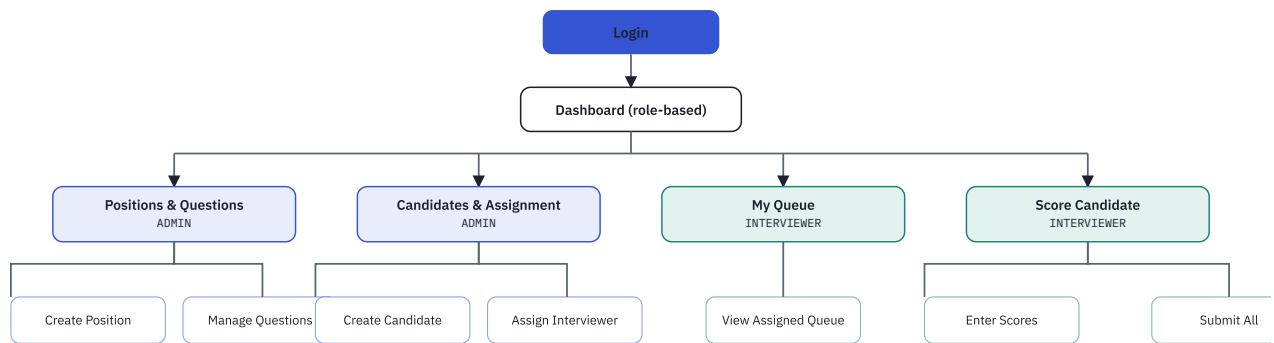


Fig 2.1 — User interface flow overview. Blue modules are admin-only; teal modules are interviewer-only, matching the role check enforced on every route.

03 • CRITERION B

User Interfaces

Draft-stage screen designs for the five v1 screens — not screenshots of a finished build.

● ● ● localhost/login

◆ Interview Platform

EMAIL

you@company.com

PASSWORD

.....

Log in

Accounts are created by an admin — no self sign-up.

Fig 1.1 — Login screen. Shared by both roles; the role on the account determines the dashboard shown next.

● ● ● localhost/admin/positions/3

Positions

Candidates

POSITION TITLE

Backend Engineer

1 :: Explain how you'd design a rate limiter

2 :: Walk through a bug you fixed under pressure

+ Add question

Fig 1.2 — Admin: Position & Questions. Order shown here is the exact order interviewers see (sequence_order).

● ● ● localhost/admin/candidates/new

FULL NAME

Jane Doe

POSITION

Backend Engineer

INTERVIEWER

S. Patel

Create Candidate

Recently created

Sam Lee

Backend Engineer · S. Patel

Not started

Fig 1.3 — Admin: Create Candidate. Exactly one Position and one Interviewer per candidate in v1.

localhost/interviewer/queue

My candidates — not started

Jane Doe

Backend Engineer

Open →

Priya Rao

Backend Engineer

Open →

Fig 1.4 — Interviewer: Queue. Filtered server-side to `interviewer_id = current user`, not just hidden in the UI.

localhost/interviewer/candidates/12

Jane Doe · Backend Engineer

1. Explain how you'd design a rate limiter

1

2

3

4

5

Optional comment

2. Walk through a bug you fixed under pressure

1

2

3

4

5

Optional comment

Fig 1.5 — Interviewer: Score Candidate. Submit sends every question's score in one request — see the Submit Scores flowchart below for what happens if one is missing or invalid.

04 • CRITERION C

Database Design — ER Diagram

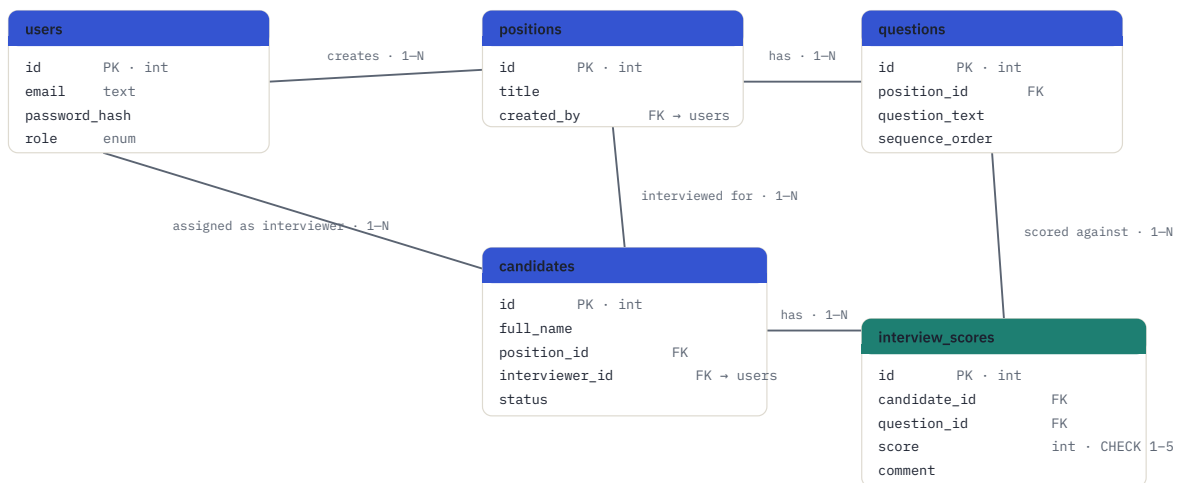


Fig 3 — Entity relationship diagram. `interview_scores` is the join point every success criterion around scoring depends on: one row per (candidate, question).

05 • CRITERION C

Data Flow Diagram — Submit Scores

The one flow every other diagram on this page ultimately supports: it's where the "no partial saves" success criterion actually gets enforced.

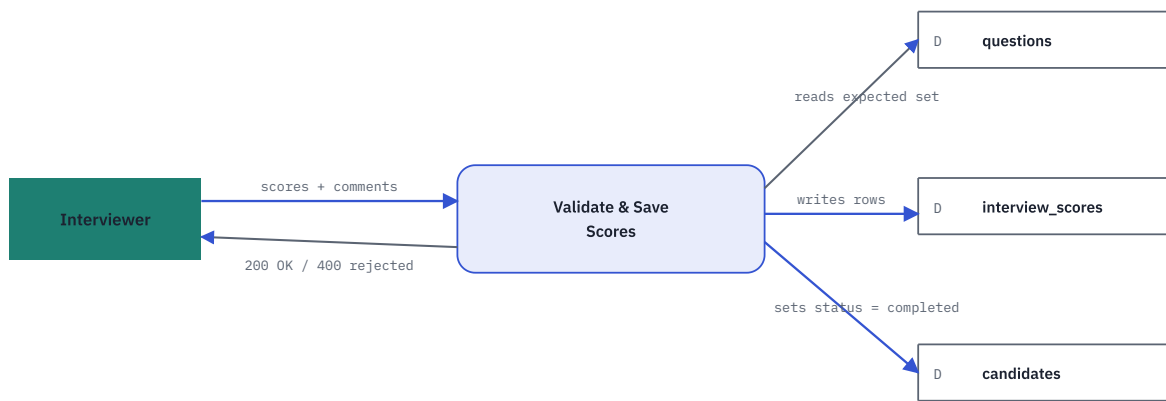


Fig 4 — Submit Scores data flow. Writes to `interview_scores` and `candidates` only happen after the read against `questions` confirms every question was answered validly.

06 • CRITERION C

Flowchart — Submit Scores

The most decision-heavy algorithm in v1, and the one the "atomic submit" and "invalid score rejected" success criteria depend on directly.

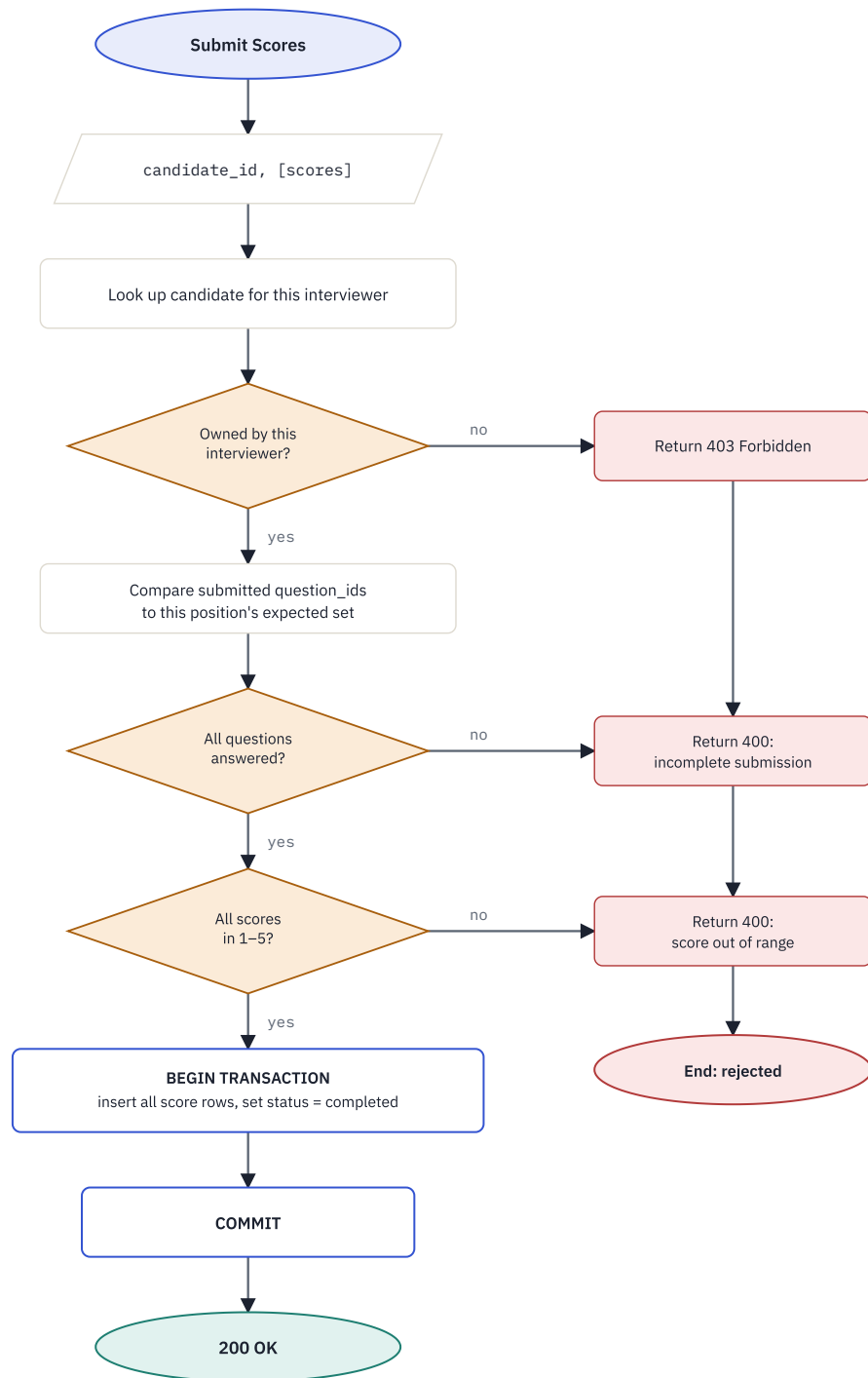


Fig 5 — Submit Scores flowchart. All validation happens before the transaction opens — nothing is written to `interview_scores` until every check passes, which is what makes the submit truly all-or-nothing.

UML Class Diagram

Backend service layer — each class wraps one module from the decomposition in `ia-documentation-v1.md`.

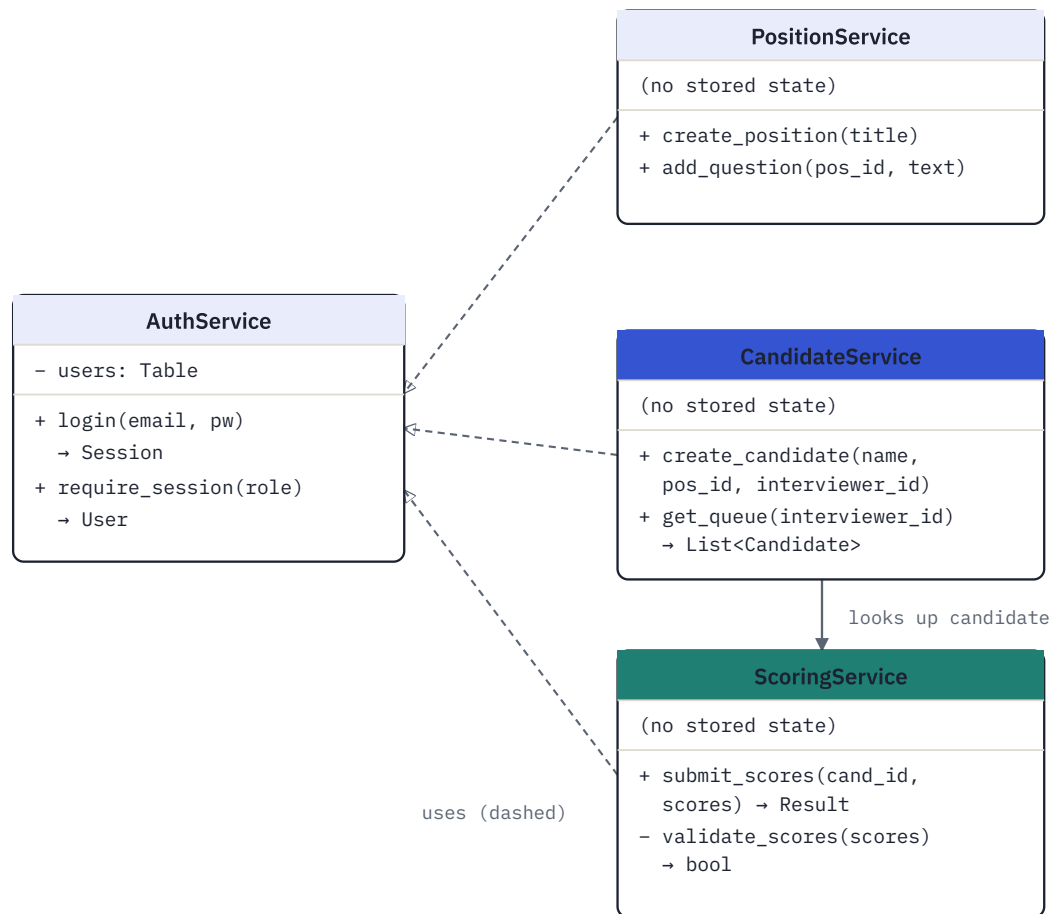


Fig 6 — UML class diagram. Every service depends on `AuthService.require_session()` — the single place role and session checks live, rather than repeated per route.

Test Plan

#	MODULE	ACTION TEST	METHOD OF TESTING	EXPECTED RESULT
1	Login	Valid credentials log the interviewer in	Submit correct email + password	200, session cookie set, dashboard loads
2	Login	Wrong password is rejected	Correct email, wrong password	401, no session set
3	Positions	Admin adds a question to a position	POST valid position_id + text	201, sequence_order auto-incremented
4	Candidates	Admin creates a candidate	POST valid position_id + interviewer_id	201, status = not_started
5	Queue	Interviewer sees only their own candidates	Interviewer B requests interviewer A's candidate	403 / not returned
6	Scoring	Interviewer submits valid scores	All questions answered, scores 1–5	200, rows written, status → completed
7	Scoring	Score outside range is rejected	One score = 0 or 6	400, nothing written (CHECK constraint)
8	Scoring	Incomplete submission is rejected	2 of 3 questions answered	400, no rows written for any question
9	Persistence	Data survives a restart	Submit scores, restart server, re-query	Same scores/status returned unchanged

09 • CRITERION C

Data Validation Tests

MODULE	FIELD	RULE	ACCEPTED	REJECTED
Add/Edit Question	question_text	Non-empty	"Explain a rate limiter"	

MODULE	FIELD	RULE	ACCEPTED	REJECTED
Add/Edit Question	sequence_order	Unique per position	1, 2, 3	duplicate order
Submit Score	score	Integer, 1–5	1, 3, 5	0, 6, "high"
Submit Score	comment	Optional text	any string, or empty	n/a – always valid
Create Candidate	position_id	Must reference an existing Position	3	999 (no such row)
Create Candidate	interviewer_id	Must reference a user with role = interviewer	interviewer's id	an admin's id
Login	email + password	Must match a stored, hashed credential	registered account	unknown email / wrong password

Companion to `docs/ia-documentation-v1.md` (Criteria A–C, extended writing). Diagram set benchmarked against a prior exemplar's Criterion B structure — user interfaces, UI flow, ER diagram, DFDs, flowcharts, UML class diagram, test plan, validation tests — scoped down to the flows that actually carry decisions in this v1 slice, rather than one diagram per CRUD action.